arXiv:2604.02141v1 [cs.GR] 2 Apr 2026

Topology-First B-Rep Meshing

YUNFAN ZHOU, New York University, USA
DANIEL ZINT, New York University, USA, TU Berlin, Germany
NAFISEH IZADYAR, University of Victoria, Canada
MICHAEL TAO, New York University, USA
DANIELE PANOZZO, New York University, USA
TESEO SCHNEIDER, University of Victoria, Canada

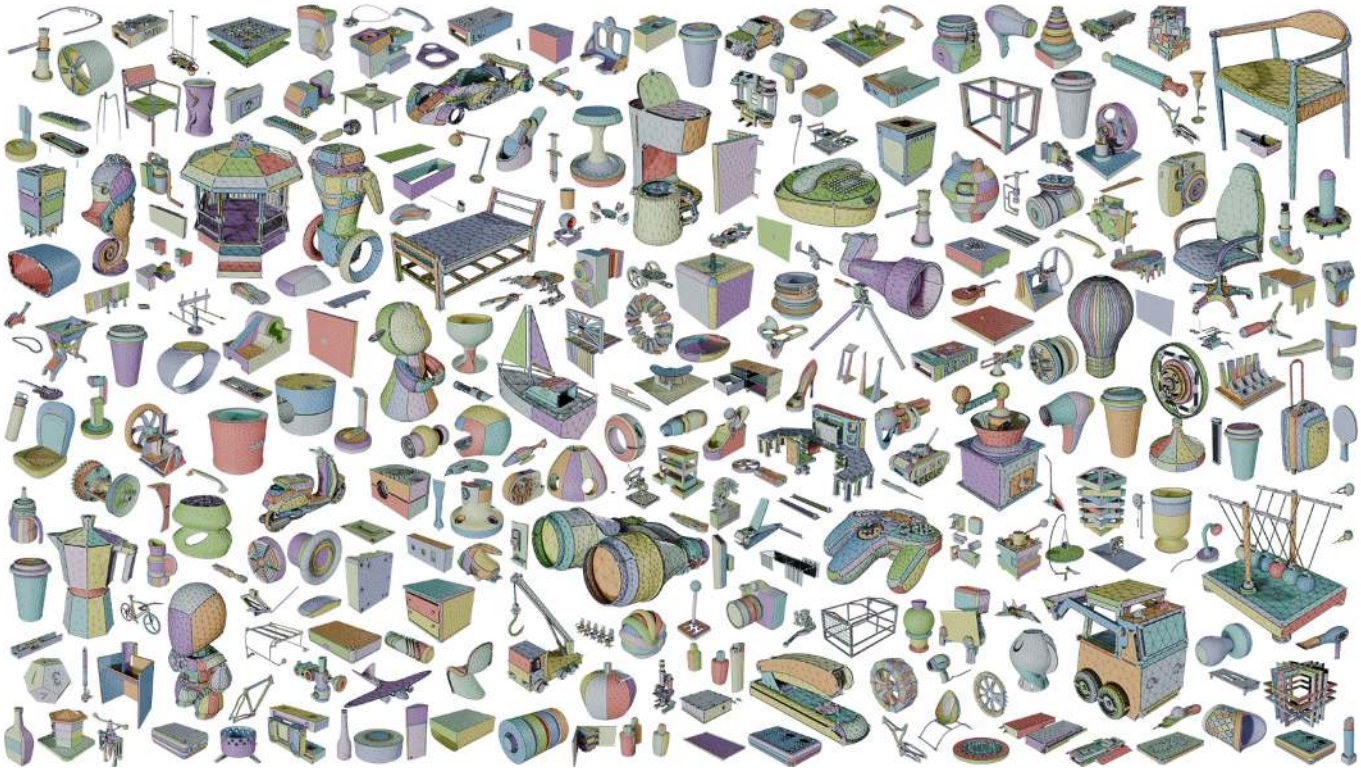


Fig. 1. Selection of models from the ABC [Koch et al. 2019] and Fusion360 [Willis et al. 2021] datasets meshed with our method; each B-Rep face is shown with a different color, illustrating preserved patch boundaries across complex models.

Parametric boundary representation models (B-Reps) are the de facto standard in CAD, graphics, and robotics, yet converting them into valid meshes remains fragile. The difficulty originates from the unavoidable approximation of high-order surface and curve intersections to low-order primitives: the resulting geometric realization often fails to respect the exact topology encoded in the B-Rep, producing meshes with incorrect or missing adjacencies. Existing meshing pipelines address these inconsistencies through heuristic feature-merging and repair strategies that offer no topological guarantees and frequently fail on complex models.

We propose a fundamentally different approach: the B-Rep topology is treated as an invariant of the meshing process. Our algorithm enforces the exact B-Rep topology while allowing a single user-defined tolerance to control the deviation of the mesh from the underlying parametric surfaces. Consequently, for any admissible tolerance, the output mesh is topologically correct; only its geometric fidelity degrades as the tolerance increases. This decoupling eliminates the need for post-hoc repairs and yields robust meshes even when the underlying geometry is inconsistent or highly approximated.

We evaluate our method on thousands of real-world CAD models from the ABC and Fusion 360 repositories, including instances that fail with standard meshing tools. The results demonstrate that topological guarantees at the algorithmic level enable reliable mesh generation suitable for downstream applications.

Authors' Contact Information: YunFan Zhou, New York University, USA; Daniel Zint, New York University, USA, and TU Berlin, Germany; Nafiseh Izadyar, University of Victoria, Canada; Michael Tao, New York University, USA; Daniele Panozzo, New York University, USA; Teseo Schneider, University of Victoria, Canada.

1 Introduction

Parametric models encoded as boundary representations (B-Reps) are the dominant representation in computer-aided design, manufacturing, and simulation. A B-Rep describes a shape through parametric primitives (patches, curves, and points) to encode geometry together with a discrete combinatorial structure, i.e., the B-Rep's topology. Patches are bounded by curves that are themselves bounded by points. The topology of a B-Rep is defined by an

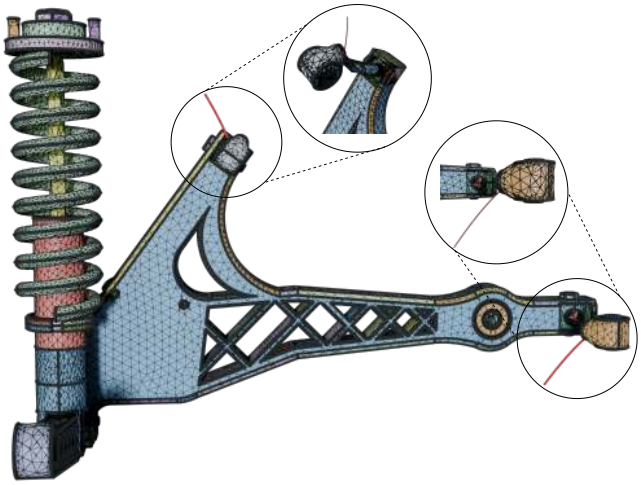


Fig. 2. Inconsistency between curves (black) and the 2D trimming curves lifted with the parameterization (red). Since our method relies only on the 3D curves (black), it successfully generates a mesh despite these inconsistencies.

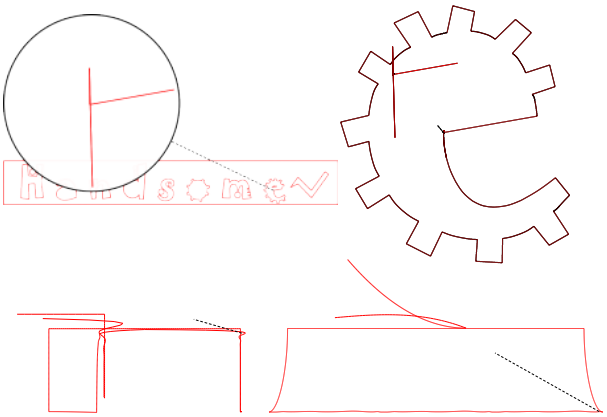


Fig. 3. Parametric domain of a patch with its 2D trimming curves. Although all curves are intended to form simple loops, approximation errors cause unintended intersections and artifacts.

adjacency graph of such primitives. While geometry and topology are tightly coupled in the representation, they play fundamentally different roles: topology encodes adjacency and incidence relationships, whereas geometry specifies how these entities are embedded in 3D as well as in parametric spaces. The geometry of curves is usually represented in two different forms: first, their parametrized 3D geometry; second, they are projected onto the parametric domain of each adjacent patch, where they serve as trimming curves. As a result, curves contain duplicated and potentially inconsistent geometric information (Figure 2). The curve geometry in the parametric domain (traditionally called trims) is obtained from the 3D geometry by inverting the patch parameterization. This inversion requires solving a non-linear optimization problem and can introduce additional errors (Figure 3); in our experiments, 14% of trims in the Fusion dataset contain intersections[1].

Multiple downstream applications, such as finite element simulation or rendering, require converting B-reps to meshes. However, this conversion remains a fragile and error-prone process. The core difficulty arises from high-order parametric patch intersections and the need to approximate them with lower-order primitives. In particular, exact intersections between parametric patches yield impractically high-order curves [Katz and Sederberg 1988a], thereby forcing systems to rely on approximations (Section 2). As a result, the geometric realization of a B-Rep often becomes inconsistent with its topology (Figure 2).

Most existing meshing pipelines attempt to address these inconsistencies by employing geometric algorithms that merge features within an ϵ-neighborhood or by applying heuristic repair strategies (Section 3). While effective for visualization and rendering, such approaches provide no guarantees on the correctness of the resulting topology and, depending on user parameters, fail to produce a mesh (Section 5.4). In practice, this results in undesired non-manifoldness, missing adjacencies, and costly downstream repairs, particularly for complex models.

In this work, we take a fundamentally different approach. Rather than resolving geometric inconsistencies heuristically, we treat the B-Rep topology as a discrete skeleton that is enforced throughout the meshing process. A key component is that we do not rely on the 2D trimming curves, as they are often flawed and may even self-intersect. Instead, we trim the parametric patches with their adjacent curves directly in 3D. Building on this insight, we propose a meshing algorithm that *guarantees preservation* of the B-Rep topology, independent of geometric approximations, while exposing a *single* geometric parameter that controls the deviation between the mesh and the underlying parametric patches (Section 4). Note that our method is guaranteed to generate meshes with the correct topology, independent of the user-specified geometric tolerance; i.e., a larger tolerance yields a less geometrically accurate mesh, but has no influence on the topology. This decoupling of topological correctness from geometric approximation enables robust mesh generation even for B-Reps with inconsistent or badly approximated geometry.

We demonstrate the robustness of our method by successfully meshing over ten thousand B-Reps drawn from real-world CAD datasets (ABC [Koch et al. 2019] and Fusion 360 [Willis et al. 2021]), including models that fail with standard meshing pipelines (Section 5.4). Our results show that enforcing topological guarantees at the algorithmic level eliminates the need for heuristic repairs and produces meshes suitable for downstream simulation and analysis.

2 Preliminaries

CAD Kernels, Feature Trees, and BReps. A *CAD kernel* is the geometric engine underlying most CAD systems: it implements the data structures and numerical algorithms used to *create, edit, and query* solid models [Mäntylä 1988; Requicha 1980]. Some popular kernels are Parasolid [Siemens Digital Industries Software 2026], ACIS/3D

[1]We detect intersections by densely sampling the trimming curves in the parametric domain.

ACIS Modeler [Corp. 2026], CGM (Convergence Geometric Modeler) [Spatial Corp. 2026], Granite (PTC Creo Granite Interoperability Kernel) [PTC 2026], Autodesk ShapeManager (ASM) [Autodesk 2026], Open CASCADE Technology (OCCT) [Open CASCADE 2026], C3D Modeler [C3D Labs 2026], and SMLib [NVIDIA 2026]. From the user's perspective, the *input* to a CAD kernel is a construction history (often a *feature tree*) that is, a sequence of operations such as extrude/revolve, fillet/chamfer, Boolean union/difference/intersection, shell/offset, or trim. The kernel evaluates this operation list and produces a final geometric model, typically stored as a *boundary representation (B-Rep)*.

B-Reps. The *output* B-Rep encodes both *geometry* (parametric curves and surfaces, often NURBS) [Piegl and Tiller 1997] and *topology* (how faces, edges, and vertices are connected) [Mäntylä 1988; Requicha 1980]. A key detail is that B-Reps faces are rarely entire parametric surfaces: instead, each face is a *trimmed patch*, i.e., a parametric surface restricted to a bounded subset of its (u, v) domain by one or more trimming curves (loops) defined in parameter space.

Geometry and Topology Inconsistencies. Crucially, by construction, the geometry and topology of a B-Rep are *not exactly consistent* except in trivial cases. Many kernel operations require computing surface–surface intersections and projecting curves between 3D space and multiple parameter domains; the exact intersection curves can have extremely high algebraic complexity even for low-degree surfaces [Katz and Sederberg 1988b]. Practical kernels, therefore, rely on approximation, subdivision, and tolerances. As a result, the same "logical" entity (e.g., a shared edge) may have multiple representations, a 3D curve plus separate trimming curves on each adjacent face, that cannot be made to coincide everywhere, leading to small gaps, overlaps, or mismatched endpoints Figure 3.

Meshing. This inconsistency is one reason a B-Rep is often *not directly usable* for downstream applications such as rendering or simulation, which typically require a discrete mesh. Extracting a valid surface or volume mesh from a B-Rep is challenging precisely because meshing must reconcile these geometric/topological mismatches: tiny gaps can create cracks, overlaps can introduce self-intersections, and local inconsistencies can break watertightness or manifoldness. Robust meshing, therefore, cannot be treated as a routine discretization step; it must explicitly handle the inherent inconsistencies of real CAD B-Reps.

Our Contribution. We propose a novel method to convert B-Reps into meshes that is unconditionally robust to these inconsistencies: our key observation is to avoid using 2D trimming curves entirely and instead reconstruct them by tracing directly in 3D over patches using closest point projection paired with a topologically robust tracing algorithm. Our contribution is orthogonal to the algorithms within a CAD kernel, and our algorithm can process B-Reps produced by any CAD kernel.

3 Related Work

B-reps come from open-source CAD kernels, like OpenCascade [Open Cascade SAS 2011] or closed-source CAD kernels,like Parasolid and ASIC, but the inherent inconsistencies of B-reps always emerge. Therefore, even if we can mesh each parametric patch [Cripps and Parwana 2011; Rockwood et al. 1989], constructing meshes with triangulated patches that are connected according to the patch topology is difficult. In this section, we discuss several approaches others have used to convert the parametric models generated by these kernels into triangulated surfaces.

Top-down methods. One class of approaches for constructing a triangle mesh from a parametric model typically involves triangulating each parametric patch and then stitching them together according to the topology of the parametric model [Cuillière 1998; Sheng and Hirsch 1992]. The boundaries of these per-patch triangulations depend on the parametric coordinates of trims, which can self-intersect (Figure 3) and therefore cannot guarantee aligned boundaries between patches, resulting in a difficult stitching process [Barequet and Kumar 1997; Kahlesz et al. 2002]. Some of these methods do not require knowledge of the input model's topology, such as [Wei and Wei 2024], which uses local matching processes to remove gaps after an initial stitching step produces invalid results. As they also rely on the trimmed patches, they are vulnerable to inconsistencies in the trimming curves (Figure 2).

Repair. Stitching per-patch meshes involves "repairing" the mesh between trims, and advances have been made to repair meshes that come from parametric models. Some methods, such as Guo et al. [2019], improve the per-surface meshes created by CAD kernels to ease the stitching process, while others, such as Wen et al. [2025], perform feature detection while repairing the full meshes generated by CAD kernels. Some have even designed user interfaces to let users manually repair gaps [Zheng et al. 2001], while others suggest repairing problematic regions by using voxel grids and implicit surface meshing to replace problematic regions [Bischoff and Kobbelt 2005; Ju 2004], or relying on user-tolerance to merge nearby patches [Busaryev et al. 2009]. Such techniques enjoy great generality, as they are applicable to triangle meshes, but because they lack the input topology, they cannot guarantee that the meshes they generate follow the original parametric model's geometry. Rather than relying on repairing inconsistencies introduced by parametric trims, our method guarantees a consistent trim that respects the topology of the input model by tracing a 3D trim directly into each patch.

Bottom-up methods. The open source softwares GMsh [Geuzaine and Remacle 2009] and OCCT [Open Cascade SAS 2011], and the method from Li et al. [2025] implement bottom-up approaches for triangulating or quadrangulating [Reberol et al. 2021] B-Reps. That is, they linearize the boundary curves between patches and then triangulate each patch to match those linearized boundaries, trivially guaranteeing that the boundaries between curves conform to one another. This triangulation process requires using the parametric trims as the boundary of a $2D$ triangulation algorithm like constrained Delaunay triangulation [Paul Chew 1989; Shewchuk 1996] or advancing front methods [Liu et al. 2024; Lo 1985; Peraire et al. 1987], but because the parametric trim curves are truncated these boundaries can be difficult to mesh due to unfortunate features like self-intersections or poor sampling (Figure 3), resulting in challenges during triangulation. Although our approach prioritizes

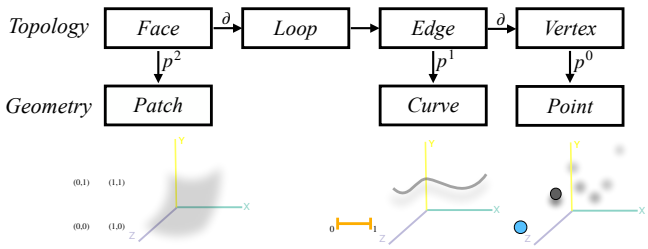


Fig. 4. A B-Rep contains topology, a combinatorial structure in which faces are bounded by loops of edges and edges by vertices, and geometry, which embeds these entities as 3D surfaces, curves, and points.

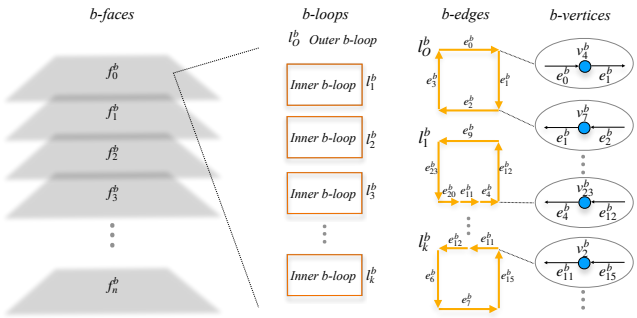


Fig. 5. Overview of the topological entities and their relationships.

data from the bottom up, we mesh our patches and curves in 3D to ensure they are sufficiently sampled, and then perform trimming in 3D, thereby sidestepping the issue of inconsistent trimming curves in the patches' parametric domains.

Adaptive trimming. Rather than putting effort into matching patches and trims, some have attempted to change the trims themselves. For example, Wang et al. [2025] propose their own method for determining trims and optimizing the resulting curves, whereas Jourdes et al. [2014] utilizes raycasting to re-generate trims by projecting the trimmed boundaries of surface A upon surface B to generate a new parametric curve on B before meshing the re-trimmed mesh of B to guarantee a conforming mesh. More globally, aligned curve meshing [Yang et al. 2025a,b; Yu et al. 2021] employs Delaunay meshing on the edge network [Xiao et al. 2021] to mesh patch boundaries. This ensures that edges do not intersect in 3D and that the network of meshed trims is topologically correct before meshing each surface. These methods still rely on constrained triangulations using trim curves in the parametric space, whereas we use only the 3D coordinates of our edges to avoid inconsistent representations across different meshes.

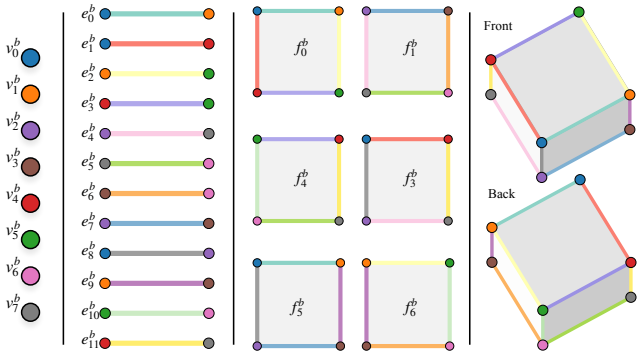


Fig. 6. B-Rep topology of a cube. The same b-vertices and b-edges are shown with the same color, highlighting the correspondence between b-vertices, b-edges, b-loops, and b-faces.

4 Method

The input to our algorithm is a boundary representation (B-Rep), consisting of a topology together with an associated geometric embedding (Figure 4). In our implementation, we use the ABS dataset [Izadyar et al. 2025], which provides B-Reps converted from STEP files. Note that models in the ABC and Fusion datasets are generated using different CAD kernels.

Topology. The topology $\mathcal{P}$ of a B-Rep is composed of a set of vertices, edges, loops, and faces (Figure 5). To distinguish them from the mesh primitives (such as triangle faces and vertices) that appear in our output later, we refer to them as *b-vertices* $\{v_i^b\}$, *b-edges* $\{e_i^b\}$, *b-loops* $\{\ell_i^b\}$, and *b-faces* $\{f_i^b\}$.

A *b-edge* is an oriented topological entity defined by an ordered pair of b-vertices (v_1^b, v_2^b) and incident to one or more b-faces. A *b-face* has a unique outer b-loop ℓ_O^b and may contain a collection of k inner b-loops $(\ell_1^b, \ldots, \ell_k^b)$ representing holes. Each *b-loop* is a closed sequence of oriented b-edges $(e_1^b, \ldots, e_n^b)$. For instance, a cube has 8 b-vertices, 12 b-edges, and 6 b-faces, all connected to obtain the correct topology (Figure 6).

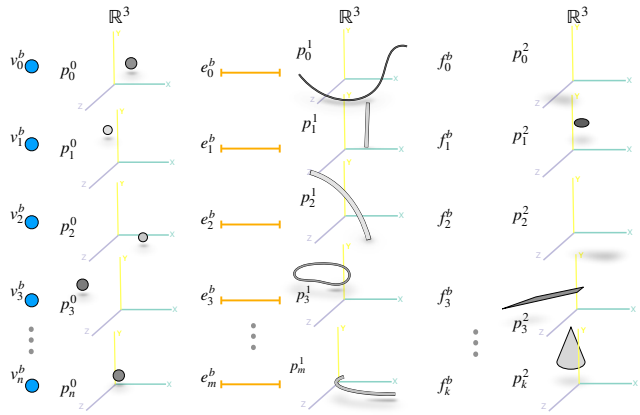


Fig. 7. Overview of the geometrical entities.

Geometry. The topological entities have their corresponding parametric shape to embed them in $\mathbb{R}^3$ (Figure 7).

- Each b-vertex v_i^b has a corresponding point $p_i^0 \in \mathbb{R}^3$.
- Each b-edge e_i^b has a parametric curve $p_i^1(t) : [0, 1] \to \mathbb{R}^3$ mapping the unit interval to a 3D curve.

- Each b-face f_i^b has a parametric surface $p_i^2(u,v)z : [0,1] \times [0,1] \to \mathbb{R}^3$ mapping the unit square $[0,1]^2$ to a 3D surface.

Note that we do not assume anything about the geometry of the topological entities. For instance, the 3D curve $p_i^1(t)$ attached to the b-edge e_i^b can be arbitrarily far from the parametric surfaces attached to the b-faces adjacent to e_i^b. In particular, the geometric embedding may be arbitrarily inconsistent with the topology (Figure 2), reflecting the need for tolerance-driven approximations present in other methods (Section 3).

Output. The output of our algorithm is a triangle mesh with the same topology as $\mathcal{P}$. A single geometric tolerance parameter ϵ controls the approximation error between the mesh and the input parametric geometry, without affecting topological correctness. We emphasize that, regardless of the input geometry, our meshing algorithm *preserves the input topology exactly*. In other words, the hierarchy of faces, loops, edges, and vertices, together with their structural relations, is reproduced *identically* in the mesh output.

4.1 Algorithm

Given a B-Rep, we first construct independent polylines and triangle meshes for each parametric curve and surface. We snap polyline endpoints to vertices and directly embed each polyline into its corresponding 3D surface mesh, thereby trimming the mesh. We then stitch the resulting trimmed surface meshes along shared b-edges to recover the full model, and finally remesh it to improve element quality while preserving topology (Figure 8). Our algorithm proceeds in five stages: (1) geometry sampling (Section 4.2), (2) curve snapping (Section 4.3),(3) loop embedding on each face (Section 4.4), (4) stitching across faces (Section 4.5), and (5) geometry- and topology-preserving isotropic remeshing (Section 4.6). Where applicable, we employ several heuristic strategies to improve performance and geometric accuracy without affecting correctness (Section 4.7).

Correctness. The base algorithm guarantees preservation of the input B-Rep topology by construction. The loop embedding stage embeds each b-loop as a simple cycle that partitions the mesh according to the known topology of the loop. The stitching stage preserves this topology by aligning boundary discretizations without introducing or removing adjacency relations. Any heuristic acceleration used in the pipeline is validated using purely topological criteria (rather than approximate geometric ones) and reverts to the base algorithm if validation fails.

4.2 Stage 1. Sampling

The goal of the sampling stage is to construct a sufficiently refined simplicial approximation of each geometric primitive of our input B-Rep. This stage is purely geometric and does not encode any topological decisions.

b-vertices. We do not need to sample the b-vertices as we already have their corresponding 3D points.

b-edges. We sample every curve p_i^1 with a polyline consisting of vertices v_i^e and edges e_i^e. Starting from a single edge, we recursively subdivide it until the distance of the polyline to the curve is less than ϵ.

b-faces. For the patch associated with a b-face f_i^b, we mesh its 2D parametric domain using MMG [Balarac et al. 2022] with the patch's induced metric, and lift the resulting triangulation to 3D via the parametric surface, obtaining the mesh M_i. Note that we mesh the entire parametric domain and ignore any trimming at this stage. Furthermore, we perform RGB subdivision [Puppo and Panozzo 2009] until the per-triangle distance between the mesh and its corresponding patch is less than ϵ and the maximum edge length is below 0.01% of the diagonal of the model. Since computing the exact distance to a parametric surface is difficult, we conservatively estimate. We sample quadrature points in each parametric triangle (order 5), lift them both using the linear (barycentric) interpolation of the mesh triangle and the parametric surface evaluation, and integrate the distance between the resulting 3D points.

4.3 Stage 2. Curve Snapping

After sampling all geometric primitives, we snap the endpoints of each sampled b-edge curve to the points corresponding to their incident b-vertices. Since the sampled endpoints generally do not coincide exactly with the b-vertex positions, snapping inevitably introduces geometric error. To avoid localized distortion, we distribute this error along the curve by applying a 1D uniform Laplacian smoothing to displacements. This step enforces consistency between the sampled curves and the B-Rep topology and ensures that curve endpoints coincide exactly with the embedded b-vertices before loop embedding.

4.4 Stage 3. Loop Embedding

This stage is performed independently for each b-face f_k^b. The goal is to embed all b-loops ℓ_i^b belonging to f_k^b, together with their b-vertices v_j^b, onto the mesh M_k of the corresponding parametric surface (Algorithm 1). Specifically, we create an edge chain on the triangle mesh M_k corresponding to each b-edge in each b-loop ℓ_i^b. During this process, we maintain correspondence between mesh edges and b-edges, as well as the association between embedded vertices and their originating b-vertices.

The core idea of our algorithm is to embed the boundary of a disk onto a disk, without relying on geometric heuristics. The parametric patch and its associated mesh M_k are topological disks; however, the input b-loops may be non-manifold and therefore may not enclose disks. To address this, we temporarily duplicate every non-manifold b-loop edge and vertex, while keeping track of the duplication (Algorithm 1, line 1). This operation yields a set of manifold b-loops, each of which encloses a topological disk, without altering the represented topology. We then embed each such disk boundary onto the disk M_k.

We begin by tracing the outer loop $\ell_O^b = \{e_i^b\}, i = 1, \ldots, n$ made by the n b-edges e_i^b (Algorithm 1, line 4). We select the first b-edge $e_1^b = (v_j^b, v_k^b)$ and project the point p_j^0 associated with the b-vertex v_j^b onto the closest triangle on the mesh M_k (Algorithm 2 line 5); we denote the resulting mesh vertex by s. Let $\{v_{1,i}^e\}_{i=1}^m$ denote the sampled points of the parametric curve associated with e_1^b. We insert the second sampled point $v_{1,2}^e$ into M_k by splitting the closest triangle to $v_{1,2}^e$ on M_k (Algorithm 2 line 7); we call this point e.

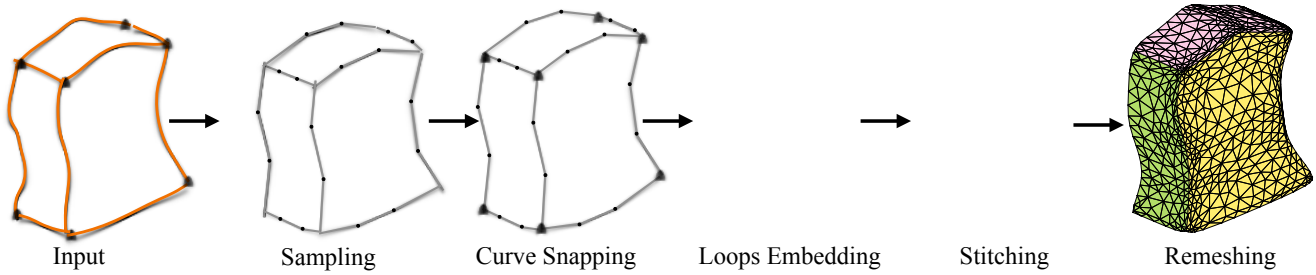


Fig. 8. Overview of our five-stage algorithm: we start by sampling every primitive, snapping curves to vertices, and embedding them on face meshes. We then stitch the trimmed meshes to obtain a topologically valid surface mesh, which we finally remesh to improve element quality.

Algorithm 1 Loop embedding for a b-face f_k^b

Input: Mesh M_k sampled from the parametric patch of f_k^b; b-loops $\{\ell_O^b, \ell_1^b, \ldots, \ell_k^b\}$ with sampled polylines
Output: Mesh M_k with all loops embedded as mesh edge chains

Preprocess non-manifold loops
Duplicate every non-manifold b-loop edge and vertex
Record a merge map for duplicated entities

Embed outer loop
$M_k \leftarrow$ EmbedLoop(M_k, ℓ_O^b, KeepDisk)

Embed inner loops
for $r \leftarrow 1$ to k **do**
  $M_k \leftarrow$ RestoreDisk(M_k)
  $M_k \leftarrow$ EmbedLoop(M_k, ℓ_r^b, KeepComplement)
end for

Restore original topology
Merge duplicated vertices and edges by collapsing recorded chains
return M_k

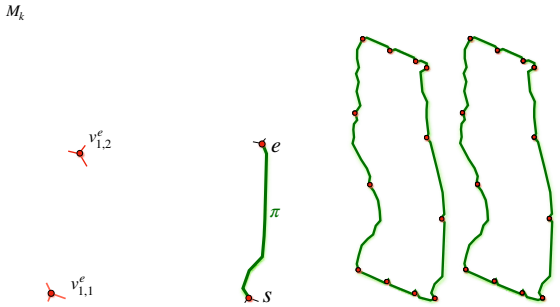


Fig. 9. Overview of our tracing algorithm: we start by projecting two points, trace an edge chain on the mesh using Dijkstra's algorithm, and repeat this process for each successive vertex.

Algorithm 2 EmbedLoop

Input: Mesh M (topological disk); b-loop $\ell = \{e_i^b\}_{i=1}^n$
Output: Mesh M with ℓ embedded as a simple cycle
1: Initialize forbidden edge set $\mathcal{F} \leftarrow \partial M$
2: Initialize embedded cycle $\Gamma \leftarrow \emptyset$
3: **for** $i \leftarrow 1$ to n **do**
4: Let $e_i^b = (v_a^b, v_b^b)$ with sampled points $\{v_{i,j}^e\}_{j=1}^m$
5: Project v_a^b onto M and denote the mesh vertex by s
6: **for** $j \leftarrow 2$ to m **do**
7: Project $v_{i,j}^e$ into M; call it e
8: $\pi \leftarrow$ shortest path from s to e on the edge graph of M avoiding $\mathcal{F}$
9: $\Gamma \leftarrow \Gamma \cup \pi$
10: $\mathcal{F} \leftarrow \mathcal{F} \cup \pi$
11: Enforce a simplicial embedding of $\mathcal{F}$ by local refinement [Zint et al. 2025]
12: $s \leftarrow e$
13: **end for**
14: **end for**
15: Cut M along Γ, yielding components (M_1, M_2)
16: **if** KeepDisk **then**
17: $M \leftarrow$ disk component among $\{M_1, M_2\}$
18: **else**
19: $M \leftarrow$ complementary component
20: **end if**
21: **return** M

To embed the first sampled segment $\{v_{1,1}^e, v_{1,2}^e\}$ of e_1^b on the mesh, we compute a shortest path π between the two embedded vertices s and e on M_k using Dijkstra's algorithm over the edges of M_k (Algorithm 2 line 8, Figure 9). To preserve the correct disk topology, boundary and previously traced edges are excluded from the graph traversal (Algorithm 2 line 10). This restriction prevents the embedded path from crossing existing boundaries and ensures consistency with the intended topological partition. We then repeat the same procedure with the next vertex $v_{1,2}^e$ until ℓ_O^b is fully embedded in M_k. Note that if the mesh is too coarse, there may not be a path between the two vertices. By making the mesh a simplicial embedding [Zint et al. 2025] of all previously traced edges, we guarantee that Dijkstra's algorithm always has a solution. Once the outer loop

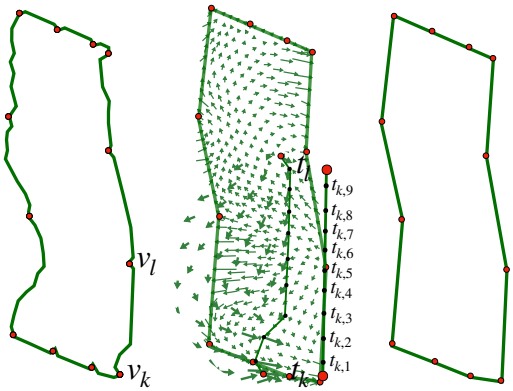


Fig. 10. Overview of snapping: after we embed the b-edge (v_k, v_l) onto the surface mesh, we snap its vertices to the target curve, and distribute the resulting snapping displacement over the mesh (green arrows), leading to a surface that conforms to the b-edge geometry.

has been fully traced, the embedded simple cycle partitions M_k into two connected components: a disk and an annulus. Since the outer b-loop is known a priori to enclose a disk, this decision is purely topological and does not rely on geometric predicates. We therefore retain the disk component and discard the annulus (Algorithm 2 line 17).

Inner-loop tracing. After embedding the outer loop, the resulting mesh remains topologically a disk and serves as the input for tracing the inner b-loops (holes) using the same procedure. Unlike the outer loop, for inner loops we discard the disk component and retain the complementary component (Algorithm 2, line 19). Once the second inner loop is traced, the mesh is no longer a disk but an annulus. To restore the disk property required for subsequent embeddings, we select one vertex on each boundary component and trace a connecting path between them [Erickson and Har-Peled 2002]. This operation is purely topological and restores the disk property without affecting the final connectivity of the mesh.

After all loops have been embedded, we restore the original B-Rep topology, which was temporarily altered to ensure manifold b-loops during embedding, by undoing the preprocessing duplication by merging duplicated entities. Duplicated b-edges are re-identified by stitching the corresponding boundary edge chains, as described in Section 4.5. Duplicated b-vertices are restored by collapsing the chains of mesh edges connecting their copies. This operation is the inverse of the initial duplication step and restores the correct B-Rep topology.

4.5 Stage 4. Stitching

The result of the previous stage is a collection of trimmed meshes, one per b-face, whose boundaries are topologically consistent but discretized with different numbers of vertices. Additionally, every boundary edge e_e^j is associated with an input b-edge e_j^b, and its endpoints have a corresponding parametric value. That is, for the edge $e_e^j = \{v_k, v_l\}$, the two vertices v_k and v_l have the associated parameters t_k and t_l on the curve attached to the b-edge e_j^b. If v_k is not the point of a b-vertex, t_k is the parametric value of the sampled point on the curve; if v_k was traced with Dijkstra's algorithm, it is arc-length interpolated from the two projected points to reduce distortion.

Fig. 11. After embedding, a b-edge has a different number of vertices on the two sides. We refine it (red) to obtain a conforming mesh.

To generate the final mesh, we snap every mesh vertex corresponding to a b-vertex to its associated b-vertex position, and every boundary vertex to the position described by its associated curve and parameter t_i. Since the points and curves are not necessarily on the mesh, the snapping introduces a localized geometric discrepancy near the boundary (Figure 10). To distribute this error, we smooth it over the mesh by computing the boundary displacement u_c and solving for

$$\Delta u = 0 \quad \text{with} \quad u|_{\partial M_k} = u_c,$$

with ∂M_k the boundary of M_k and Δ the uniform Laplace operator, as cotangent weights may become unstable on low-quality or highly anisotropic meshes. Note that triangles may contain two vertices on ∂M_k without the corresponding edge lying on the boundary; therefore, before assembling and solving the Laplace equation, we ensure that M_k is a simplicial embedding of its boundary ∂M_k. Finally, to stitch the meshes together, we iterate over every b-edge e_j^b, find the b-faces attached by it, and "align" the boundary edges of the two meshes. That is, we sort the union of the parametric values from both sides, and insert any missing vertex so that the two meshes are conforming on the boundary (Figure 11). This guarantees that adjacent meshes are conforming and share identical discretizations along each b-edge. As a result, the stitched mesh is globally conforming: each b-edge corresponds to a single shared polyline in the final mesh. Note that when stitching edges from the same patch, the resulting mesh may become non-manifold if it is too coarse near the boundary. To prevent this issue, we refine the affected mesh edges twice, ensuring that there are always at least three edges between any pair of boundary edges.

4.6 Stage 5. Remeshing

The result of the previous stages is a mesh whose topology exactly matches the input B-Rep. However, during projection and stitching, additional vertices may be inserted, leading to irregular element sizes and reduced mesh quality (Figure 12, left). To improve mesh quality, we apply isotropic remeshing [Botsch and Kobbelt 2004] with several targeted modifications.

Topology Preservation. To preserve our topological guarantees, we restrict remeshing operations to prevent any changes in mesh

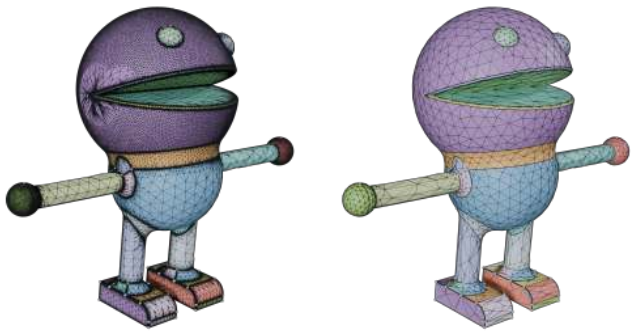


Fig. 12. Example of a mesh before (left) and after remeshing (right).

connectivity across b-edges, b-loops, and b-faces. Specifically, using the declarative specification framework [Jiang et al. 2022], we freeze b-vertices, prevent edge swaps on b-edges, and allow edge collapses only on internal edges or along b-edges (but never between them).

Geometry Accuracy. Inspired by TetWild [Hu et al. 2020, 2018], we constrain vertex motion using two geometric envelopes: one enforcing proximity to the parametric surfaces associated with b-faces, and one enforcing proximity to the 3D curves associated with b-edges. This ensures that the remeshed surface remains within a user-specified tolerance of the input geometry.

Surface Smoothing. Isotropic remeshing typically relies on tangential smoothing to improve element quality while avoiding surface shrinkage. In our setting, since vertices must remain on the input geometry, we first apply classical Laplacian smoothing and then project onto the corresponding parametric surface. This effectively mimics tangential smoothing on the input surface while respecting the geometric envelopes.

Quality Optimization. Isotropic remeshing aims to produce regular meshes, often favoring vertex valences close to 6 by performing edge flips. In our case, rather than enforcing a target valence, we explicitly evaluate triangle shape regularity [Bank and Smith 1997] before and after each edge flip and accept the operation only if it improves it.

Degenerate Triangles. To prevent the creation of degenerated triangles, we disallow any operation that would introduce triangles with zero area.

Fold-overs. We prevent fold-overs by rejecting an edge flip if it increases the dihedral angle between the adjacent to more than 120° by checking if the dot product of the two normals is less than -0.5.

4.7 Heuristic

The previously described algorithm produces a mesh with the same topology as the input B-Rep; however, the embedding procedure, the subsequent projections, and stitching may deviate significantly from the B-Rep geometry. To improve both runtime and geometric accuracy, we introduce several heuristic accelerations (Section 5.2). These heuristics do not affect correctness: if either heuristic fails its validation checks, we automatically revert to the topology-preserving correct algorithm. Note that all heuristic outputs are validated solely on topological criteria before acceptance.

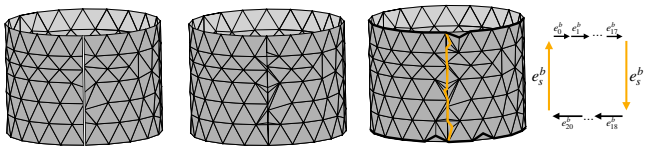


Fig. 13. Handling periodic patches. We first stitch the mesh (double lines) to form a conforming periodic surface. We then trace b-edges as usual, while avoiding tracing repeated b-edges (orange) more than once.

Initial refinement. To ease the tracing of b-edges (Section 4.4), we apply an additional RGB subdivision. For every b-edge e_i^b attached to a b-face f_j^b, we upsample its associated curve using 0.01ϵ, with at least 10 sample points. All upsampled points are then projected to their closest points on the mesh M_j, and we apply RGB subdivision until each triangle contains at most one projected sample point.

Singular patches. If the patch has a singularity on the boundary of the parametric domain, we collapse all boundary edges to a single point. This allows us to create meshes with low geometric error for shapes such as spheres and cones.

Optimistic tracing. During tracing of inner loops, we first try to embed them without connecting existing loops to the boundary, assuming that the embedded loop partitions the mesh into two connected components, one of which is topologically a disk. If this condition is satisfied, the trace is accepted and the algorithm proceeds. If the embedded cycle does not yield such a partition, we reject the trace and apply the standard algorithm.

Outer loop tracing. In our experience, most outer loops coincide with the boundary of the parametric patch. For this reason, embedding the outer loop while excluding boundary edges can introduce large geometric distortion and unnecessary refinement. To limit this issue, we allow the outer loop to use boundary edges during tracing. After each vertex is connected, we verify that the mesh remains a single connected component, as the trace may inadvertently connect two distinct boundary edges of the meshed patch. If this condition is violated, we project all remaining points, discard the component with the fewest projected points, and continue tracing on the remaining component using the standard procedure. Once the outer loop has been fully traced, we verify that the resulting mesh is topologically a disk; if this check fails, we revert and use the default outer-loop tracing algorithm.

Periodic patches. Since we rely on closest-point projection, when two regions of M_k are close in $\mathbb{R}^3$ but far apart along the surface or not adjacent in the mesh connectivity, as is common for periodic b-faces, the projection may alternate between geometrically nearby but topologically distant locations, producing a "zig-zag" (Figure 16, bottom). Additionally, repeatedly tracing identical b-edges can lead to unnecessary mesh refinement and poor geometric quality. We therefore devise a heuristic that avoids retracing vertices and edges that have already been embedded. If the parametric surface associated with the b-face is periodic, we first modify the topology of

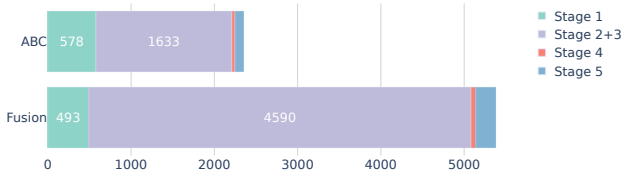


Fig. 14. Average runtime distribution of the individual stages for the two datasets.

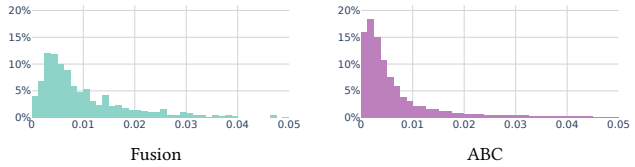


Fig. 15. Histogram of the geometric error relative to the bounding box diagonal.

M_k to match the surface periodicity (Figure 13). We then trace the outer loop as usual, except that repeated b-edges are traced only once. After tracing, we require that the retained component is topologically an open disk. If this condition is not satisfied, we revert the trace, restore the original disk topology of M_k, and trace the outer loop using the standard algorithm. Note that this heuristic is applied only to outer loops, since after tracing the outer loop, the remaining component is a disk.

Long traces. During loop tracing, if the length of the embedded edge-chain exceeds the length of the corresponding b-edge by more than a factor of 100, we stop the trace and resample the patch with a smaller tolerance. This heuristic is used purely for performance: when the mesh is too coarse, no sufficiently close path may exist on the surface. Refining the mesh increases resolution and typically enables a shorter, more accurate trace.

5 Results

Our algorithm is implemented in Python and C++, and uses the Wildmeshing-toolkit [Jiang et al. 2022] for mesh data structures and editing operations. All experiments were run on a single cluster node equipped with an Intel Cascade Lake Platinum 8268 and Xeon Platinum 8592 processor, limited to one thread, 100 GB of RAM, and a maximum runtime of 8 hours per model.

5.1 Datasets and experimental setup

We evaluate our method on real-world CAD data drawn from two datasets: one chunk of the Fusion360 [Willis et al. 2021] dataset (approximately 750 models) and one chunk of the ABC [Koch et al. 2019] dataset (approximately 10 000 models). Since both datasets provide models in STEP format, we use the B-Rep representations from the ABS dataset [Izadyar et al. 2025], which contains a one-to-one conversion of STEP files into an HDF5 format suitable for processing. When a model contains multiple parts, we mesh only the first part. These datasets contain complex B-Reps with a wide range of geometric and topological configurations, including thin features, small tolerances, and inconsistent geometric embeddings. Importantly, the data originates from different CAD kernels: Fusion360 uses its own kernel, while ABC models are sourced from OnShape, which uses the Parasolid kernel.

Only 317 (i.e., around 3%) models did not terminate within the imposed time and memory limits. In all such cases, increasing the available resources allowed the computation to complete successfully, indicating that failures are due to resource limits rather than algorithmic breakdowns.

Runtime. Figure 14 reports the runtimes of the individual stages for the two datasets. Stages 2 and 3 were grouped, as the runtime of stage 2 is negligible. Fusion360 models generally require longer runtimes due to their greater geometric and topological complexity. A breakdown of the runtime shows that loop embedding dominates the overall cost (70% and 85% of the total runtime for ABC and Fusion, respectively), followed by sampling (25% and 10%), while stitching and remeshing contribute marginally.

Geometric accuracy. Although our method does not guarantee geometric optimality (snapping operations introduce local distortion), it consistently produces meshes that closely approximate the input geometry. Figure 15 shows a histogram of the geometric error normalized by the bounding box diagonal. More than 93% of the B-Rep models exhibit errors below 3%, whereas only 1% exhibit deviations larger than 10% up to 38%.

5.2 Ablation study

To evaluate the impact of our heuristic accelerations, we perform an ablation study by disabling them. We compare the runtime and results with and without the heuristic. Disabling heuristics significantly lowers geometric accuracy, particularly for models with periodic faces or closely spaced surface regions, while having little effect on topological correctness. With the whole pipeline, our method successfully generates a high-quality mesh (Figure 16).

5.3 Robustness

Scale variation. Our method successfully handles models with extreme scale variation, thin features, and complex loop structures. Figure 17 shows a large-scale engineering model containing fine details spanning several orders of magnitude, which is correctly meshed by our method without manual intervention.

Topology. Our method robustly handles non-manifold loop configurations commonly found in real B-Rep data (Section 4.4). Figure 18 illustrates an example in which a single b-vertex participates in multiple loops on the same patch. Our method enables processing of any model, regardless of b-loop topology, without relying on geometric repair heuristics.

Geometry. In the evaluated datasets, we encounter models whose parametric domains have extremely small area (below 10^{-14}) or are degenerate. Such cases are particularly challenging for methods that rely on geometric tolerances or robust inversion of the parameterization. Our method remains robust for these cases. To avoid numerical issues during initial surface sampling, we pad the

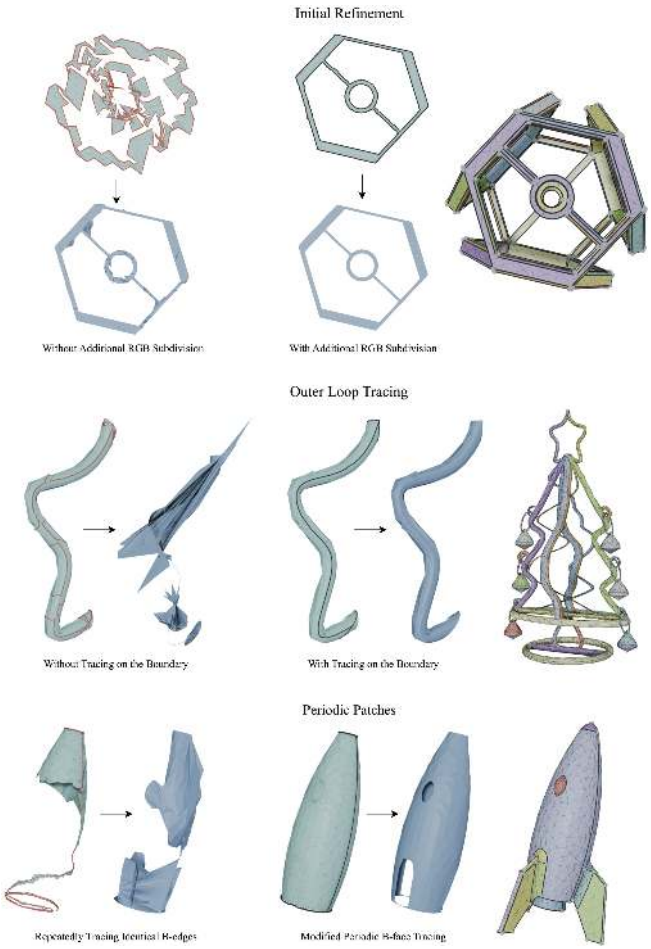


Fig. 16. Example of a B-Rep model run with and without heuristics. Both meshes are topologically valid; however, using the full pipeline yields a more geometrically accurate model.

parametric domain by a small amount (10^{-10}) before meshing. This padding enlarges the domain just enough to enable the construction of an initial triangle mesh without altering the intended topology. All subsequent stages of the algorithm operate unchanged, and the padding does not affect the final mesh connectivity or correctness. Figure 19 shows an example of a model with near-degenerate parametric domains that is successfully meshed by our method.

5.4 Comparison with existing methods

To the best of our knowledge, OpenCascade [Open Cascade SAS 2011], Gmsh [Geuzaine and Remacle 2009], NetGen [Schöberl 1997], and Mefisto [MEFISTO 2024] are the only open-source systems capable of directly meshing B-Reps in STEP format. Note that OpenCascade fails to generate meshes for 1.56% of models in the ABC dataset and 8.82% of models in the Fusion dataset [Izadyar et al. 2025]. All methods require user-defined parameters; for each, we evaluate three configurations: coarse, default, and fine.

Let ℓ denote the diagonal length of the model's bounding box. For all methods, we define three parameter regimes (coarse, default, and fine) by decreasing or increasing a parameter by a factor of 5 relative to the default. For Gmsh, we modify the min/max element size: the default setting is $[0, 10^{22}]$, the coarse setting is $[0.25\ell, 10^{22}]$, and the fine setting is $[0, 0.01\ell]$. For OpenCascade, we vary the linear deflection: the default is $10^{-3}\ell$, the coarse setting is $5 \cdot 10^{-3}\ell$, and the fine setting is $2 \cdot 10^{-4}\ell$. NetGen (via FreeCAD) provides predefined qualitative resolution settings; we use *Moderate* as the default, *Very Coarse* for coarse, and *Very Fine* for fine. Mefisto (via FreeCAD) automatically estimates an optimal edge length l, which we use as the default setting. For the coarse setting we use $5l$, while for the fine setting we use $l/5$. For our method, we vary the target edge length: the default is 0.05ℓ, the coarse setting is 0.25ℓ, and the fine setting is 0.01ℓ.

Existing methods frequently fail on challenging models (Figure 20). OpenCascade is sensitive to parameter choices and often misses features (e.g., the funnel of the coffee grinder model). Gmsh fails to capture thin structures at default resolution and runs out of memory on complex models (e.g., the lamp). Mefisto performs better overall but produces incomplete meshes at low and default resolutions. NetGen failed to generate valid meshes for most of the tested models. In contrast, our method succeeds on all tested models, consistently preserving all B-Rep faces, loops, and adjacencies.

We also tested all methods on a simpler, smoother model (the seahorse), for which all methods generated a mesh. At coarse resolution, Gmsh fails to capture curve features, while the other methods generate low-quality meshes. With our method, features are consistently captured at all resolutions, and the resulting meshes exhibit higher quality.

5.5 Downstream applications

The resulting meshes can be refined directly using standard surface-refinement techniques (Figure 21). Additionally, our models can be used directly within a simulation pipeline in which boundary conditions are specified on B-Rep faces and automatically transferred to a tetrahedral mesh. We generate a volumetric mesh using TetGen [Si 2015] and propagate B-Rep face identifiers from our surface mesh to the tetrahedral boundary, enabling direct application of boundary conditions without any manual cleanup or retagging (Figure 22).

6 Conclusion

We presented a novel approach to convert B-Reps into triangle meshes that preserves the B-Rep topology. We compared with existing algorithms, demonstrating their superior reliability and consistent quality in the generated meshes.

These properties lead to higher computational cost than competing methods: we believe this is a worthwhile tradeoff for use cases that favor automation over running time.

There are two main avenues of work that we believe are interesting: (1) we wonder if similar topology first approaches could also be applied to the CAD kernel itself, where the current numerical methods used to compute intersections and trims could be complemented by a stronger topological prior (for example, the genus of the intersection between two patches has to be 1) to obtain more

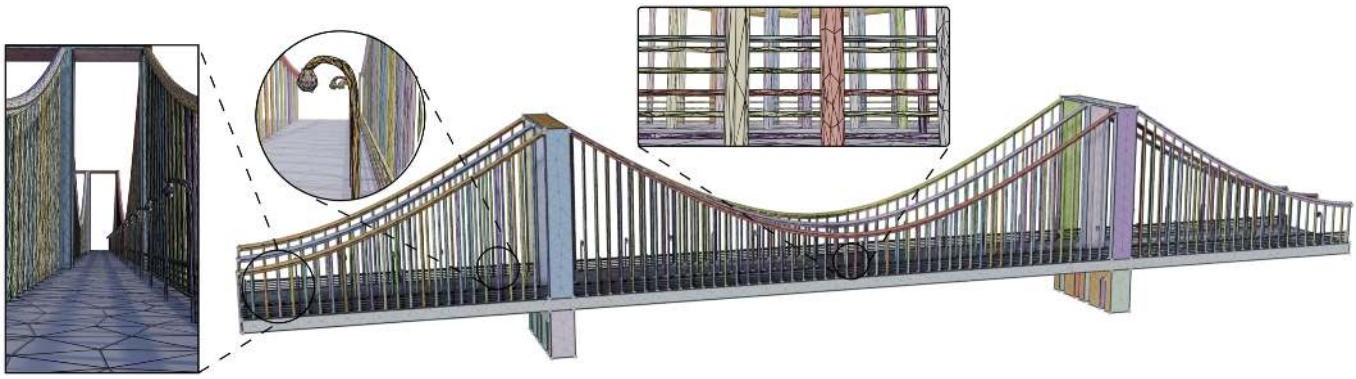


Fig. 17. A large B-Rep model with fine details is correctly meshed by our method.

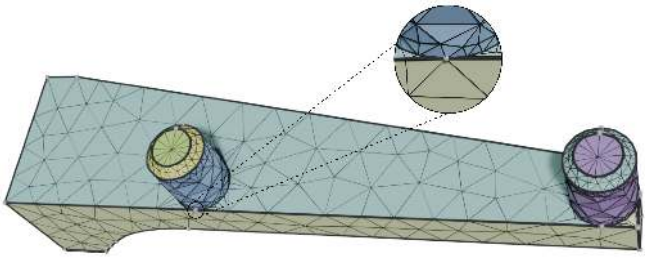


Fig. 18. B-Rep with non-manifold b-loops successfully meshed by our method.

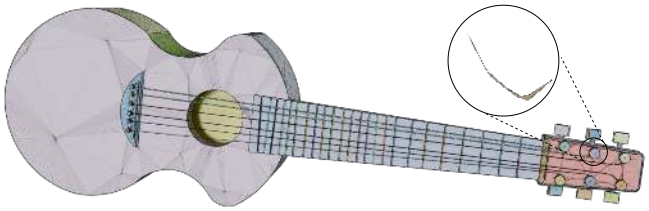


Fig. 19. B-Rep with extremely small parametric domains (close-up) successfully meshed by our method. Although several patches have nearly degenerate parameterizations, the resulting surface mesh remains valid and topologically correct.

reliably implementation of CAD kernel operations. (2) We currently only track correspondences between the edge features, but it would be possible to propagate this information to the patch interiors using a bijective parametrization approach, to obtain dense correspondences between the B-Rep and its mesh counterpart. This could be useful for inverse design algorithms or for transferring physical quantities computed on the triangle mesh back to the B-Rep.

Although our method provides strong topological guarantees, it offers only best-effort preservation of geometry and does not provide formal geometric guarantees, even if the observed errors are small in practice (Figure 15). If the input geometry is inconsistent or ill-posed, our algorithm will still produce a mesh, but the resulting geometry may be poor. Investigating how to integrate explicit geometric guarantees into our pipeline is an important direction for future work. Similarly, when the input topology is itself pathological, we do not attempt to repair it, but instead, we faithfully preserve it.

References

Autodesk. 2026. Autodesk to Develop Next Generation Solid Modeling Kernel to Improve Design Productivity. https://investors.autodesk.com/news-releases/news-release-details/autodesk-develop-next-generation-solid-modeling-kernel-improve. Accessed: 2026-01-19.

G. Balarac, F. Basile, P. Bénard, F. Bordeu, J.-B. Chapelier, L. Cirrottola, G. Caumon, C. Dapogny, P. Frey, A. Froehly, G. Ghigliotti, R. Laraufie, G. Lartigue, C. Legentil, R. Mercier, V. Moureau, C. Nardoni, S. Pertant, and M. Zakari. 2022. Tetrahedral remeshing in the context of large-scale numerical simulation and high performance computing. *MathematicS In Action* 11, 1 (2022), 129–164. doi:10.5802/msia.22

Randolph E. Bank and R. Kent Smith. 1997. Mesh Smoothing Using A Posteriori Error Estimates. *SIAM J. Numer. Anal.* 34, 3 (1997), 979–997. arXiv:https://doi.org/10.1137/S0036142994265292 doi:10.1137/S0036142994265292

Gill Barequet and Subodh Kumar. 1997. Repairing CAD models. In *Proceedings of the 8th Conference on Visualization '97* (Phoenix, Arizona, USA) *(VIS '97)*. IEEE Computer Society Press, Washington, DC, USA, 363–ff.

Stephan Bischoff and Leif Kobbelt. 2005. Structure Preserving CAD Model Repair. *Computer Graphics Forum* 24, 3 (2005), 527–536. doi:10.1111/j.1467-8659.2005.00878.x

Mario Botsch and Leif Kobbelt. 2004. A remeshing approach to multiresolution modeling. In *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing*. 185–192.

Oleksiy Busaryev, Tamal K. Dey, and Joshua A. Levine. 2009. Repairing and meshing imperfect shapes with Delaunay refinement. In *2009 SIAM/ACM Joint Conference on Geometric and Physical Modeling (SIAM '09)*. ACM, 25–33. doi:10.1145/1629255.1629259

C3D Labs. 2026. C3D Modeler: Geometric Modeling Kernel. https://c3dlabs.com/products/c3d-toolkit/modeler/. Accessed: 2026-01-19.

Spatial Corp. 2026. 3D ACIS Modeler. https://www.spatial.com/solutions/3d-modeling/3d-acis-modeler. Accessed: 2026-01-19.

R.J. Cripps and S.S. Parwana. 2011. A robust efficient tracing scheme for triangulating trimmed parametric surfaces. *Computer-Aided Design* 43, 1 (Jan. 2011), 12–20. doi:10.1016/j.cad.2010.08.009

JC Cuillière. 1998. An adaptive method for the automatic triangulation of 3D parametric surfaces. *Computer-Aided Design* 30, 2 (1998), 139–149. doi:10.1016/S0010-4485(97)00085-7

Jeff Erickson and Sariel Har-Peled. 2002. Optimally cutting a surface into a disk. In *Proceedings of the Eighteenth Annual Symposium on Computational Geometry (SCG '02)*. Association for Computing Machinery, 244–253.

Christophe Geuzaine and Jean-François Remacle. 2009. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *Internat. J. Numer. Methods Engrg.* 79, 11 (May 2009), 1309–1331. doi:10.1002/nme.2579

Jianwei Guo, Fan Ding, Xiaohong Jia, and Dong-Ming Yan. 2019. Automatic and high-quality surface mesh generation for CAD models. *Computer-Aided Design* 109 (April 2019), 49–59. doi:10.1016/j.cad.2018.12.005

Yixin Hu, Teseo Schneider, Bolun Wang, Denis Zorin, and Daniele Panozzo. 2020. Fast tetrahedral meshing in the wild. *ACM Transactions on Graphics (TOG)* 39, 4 (2020), 117–1.

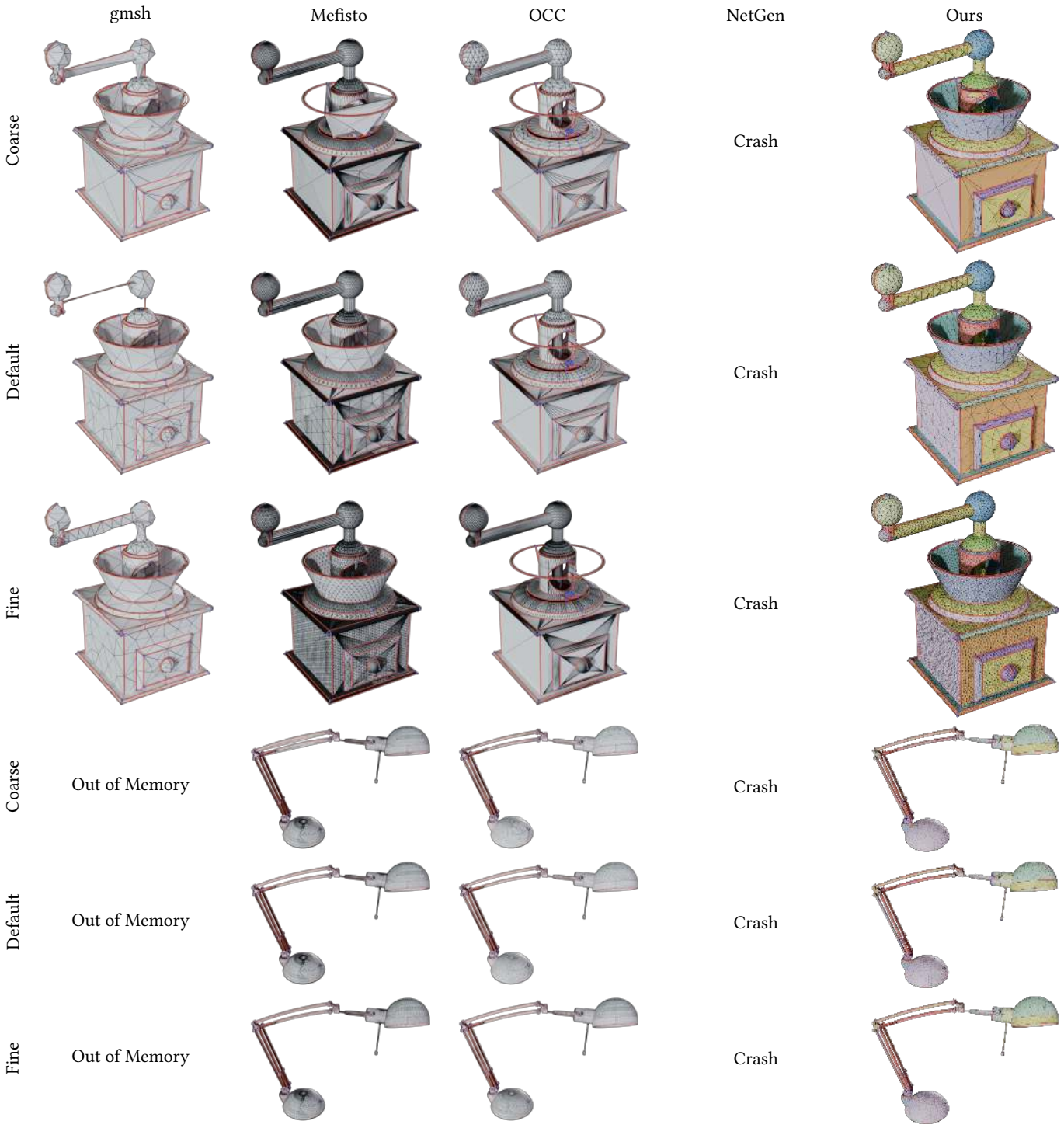
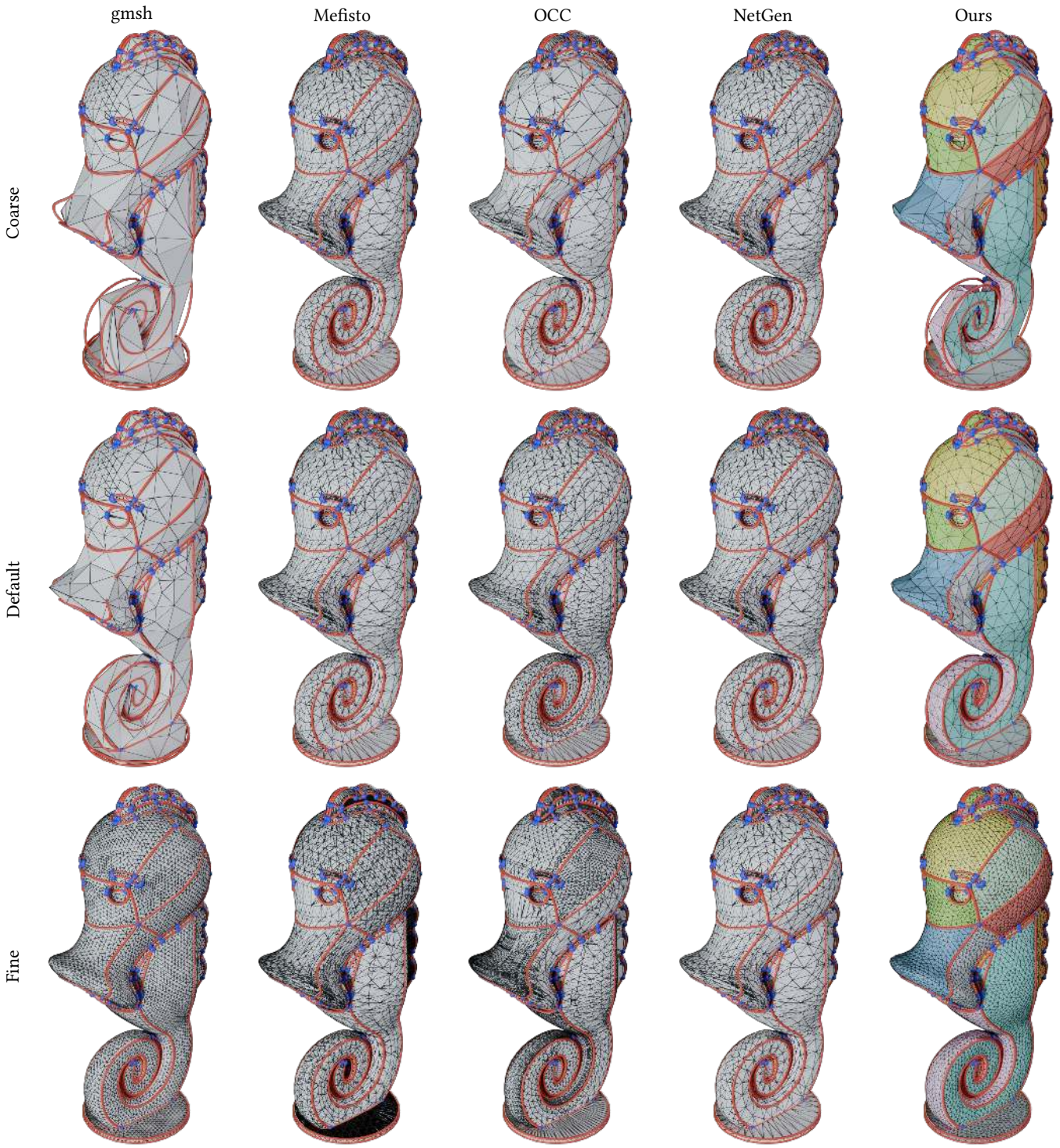


Fig. 20. Comparison of B-Rep meshing results across different methods and resolutions. Columns correspond to Gmsh, Mefisto, OpenCascade (OCC), NetGen, and our method. Rows show three parameter regimes (coarse, default, and fine). Failed runs are explicitly indicated.



Yixin Hu, Qingnan Zhou, Xifeng Gao, Alec Jacobson, Denis Zorin, and Daniele Panozzo. 2018. Tetrahedral meshing in the wild. *ACM Trans. Graph.* 37, 4 (2018), 60–1.

Nafiseh Izadyar, Sai Chandra Madduri, and Teseo Schneider. 2025. Better STEP, a format and dataset for boundary representation. arXiv:2506.05417 [cs.CV] https://arxiv.org/abs/2506.05417

Zhongshi Jiang, Jiacheng Dai, Yixin Hu, Yunfan Zhou, Jeremie Dumas, Qingnan Zhou, Gurkirat Singh Bajwa, Denis Zorin, Daniele Panozzo, and Teseo Schneider. 2022. Declarative Specification for Unstructured Mesh Editing Algorithms. *ACM Trans. Graph.* 41, 6, Article 251 (Nov. 2022), 14 pages. doi:10.1145/3550454.3555513

Fig. 21. Examples of application for our coarse meshes. A mesh (left) can be refined (right) to increase its quality.

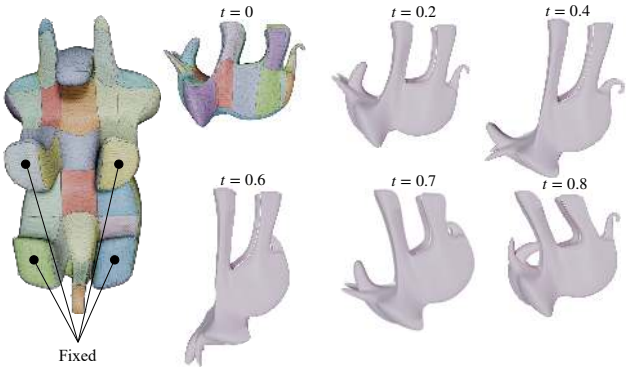


Fig. 22. An input B-Rep is transformed into a simulation-ready tetrahedral mesh (left) and used directly in a finite element simulation with boundary conditions defined on B-Rep faces (right).

François Jourdes, Georges-Pierre Bonneau, Stefanie Hahmann, Jean-Claude Léon, and François Faure. 2014. Computation of components' interfaces in highly complex assemblies. *Computer-Aided Design* 46 (Jan. 2014), 170–178. doi:10.1016/j.cad.2013.08.029

Tao Ju. 2004. Robust repair of polygonal models. *ACM Trans. Graph.* 23, 3 (Aug. 2004), 888–895. doi:10.1145/1015706.1015815

Ferenc Kahlesz, Ákos Balázs, and Reinhard Klein. 2002. Multiresolution rendering by sewing trimmed NURBS surfaces. In *Proceedings of the seventh ACM symposium on Solid modeling and applications (SM02)*. ACM, 281–288. doi:10.1145/566282.566323

Sheldon Katz and Thomas W. Sederberg. 1988a. Genus of the Intersection Curve of Two Rational Surface Patches. *Computer Aided Geometric Design* 5, 3 (1988), 253–258. doi:10.1016/0167-8396(88)90006-4

Sheldon Katz and Thomas W. Sederberg. 1988b. Genus of the intersection curve of two rational surface patches. *Computer Aided Geometric Design* 5, 3 (1988), 253–258. doi:10.1016/0167-8396(88)90006-4

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. 2019. ABC: A Big CAD Model Dataset For Geometric Deep Learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Gangyi Li, Zhongxuan Luo, Yuhao Feng, Lingfeng Zhang, Yuqiao Gai, and Na Lei. 2025. Robust tessellation of CAD models without self-intersections. *Journal of Computational Design and Engineering* 13, 1 (Dec. 2025), 239–258. doi:10.1093/jcde/qwaf134

Taoran Liu, Hongfei Ye, Jianjing Zheng, Yao Zheng, and Jianjun Chen. 2024. Advancing Front Mesh Generation on Dirty Composite Surfaces. *Computer-Aided Design* 169 (April 2024), 103683. doi:10.1016/j.cad.2024.103683

S. H. Lo. 1985. A new mesh generation scheme for arbitrary planar domains. *Internat. J. Numer. Methods Engrg.* 21, 8 (Aug. 1985), 1403–1426. doi:10.1002/nme.1620210805

Martti Mäntylä. 1988. *An Introduction to Solid Modeling*. Computer Science Press, College Park, MD.

MEFISTO. 2024. Mefisto mesh generation and finite element software. https://www.ljll.fr/perronnet/mefistoa.charger.php Open source mesh generation and FEM toolkit supporting surface and volume meshes from CAD primitives.

NVIDIA. 2026. SMLib Introduction. https://docs.nvidia.com/smlib/manual/smlib/introduction/index.html. Accessed: 2026-01-19.

Open CASCADE. 2026. Open CASCADE Technology: Collaborative Development Portal. https://dev.opencascade.org/. Accessed: 2026-01-19.

Open Cascade SAS. 2011. *Open CASCADE Technology*. https://dev.opencascade.org

L. Paul Chew. 1989. Constrained delaunay triangulations. *Algorithmica* 4, 1–4 (June 1989), 97–108. doi:10.1007/bf01553881

J Peraire, M Vahdati, K Morgan, and O.C Zienkiewicz. 1987. Adaptive remeshing for compressible flow computations. *J. Comput. Phys.* 72, 2 (Oct. 1987), 449–466. doi:10.1016/0021-9991(87)90093-3

Les Piegl and Wayne Tiller. 1997. *The NURBS Book* (2 ed.). Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-59223-2

PTC. 2026. The PTC Creo Granite Interoperability Kernel. https://support.ptc.com/images/cs/articles/2020/06/1593411762dEcH/PTC_Creo_Granite_Interoperability_Kernel._Final.pdf. Accessed: 2026-01-19.

Enrico Puppo and Daniele Panozzo. 2009. RGB Subdivision. *IEEE Transactions on Visualization and Computer Graphics* 15, 2 (2009), 295–310. doi:10.1109/TVCG.2008.87

Maxence Reberol, Christos Georgiadis, and Jean-François Remacle. 2021. Quasi-structured quadrilateral meshing in Gmsh - a robust pipeline for complex CAD models. *CoRR* abs/2103.04652 (2021). arXiv:2103.04652 https://arxiv.org/abs/2103.04652

Aristides G. Requicha. 1980. Representations for Rigid Solids: Theory, Methods, and Systems. *Comput. Surveys* 12, 4 (Dec. 1980), 437–464. doi:10.1145/356827.356833

Alyn Rockwood, Kurt Heaton, and Tom Davis. 1989. Real-time rendering of trimmed surfaces. *ACM SIGGRAPH Computer Graphics* 23, 3 (July 1989), 107–116. doi:10.1145/74334.74344

Joachim Schöberl. 1997. NETGEN An advancing front 2D/3D-mesh generator based on abstract rules. In *Computing and Visualization in Science*. Vol. 1. Springer, 41–52.

X. Sheng and B.E. Hirsch. 1992. Triangulation of trimmed surfaces in parametric space. *Computer-Aided Design* 24, 8 (Aug. 1992), 437–444. doi:10.1016/0010-4485(92)90011-x

Jonathan Richard Shewchuk. 1996. *Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator*. Springer Berlin Heidelberg, 203–222. doi:10.1007/bfb0014497

Hang Si. 2015. TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator. *ACM Trans. Math. Software* 41, 2 (Feb. 2015), 1–36. doi:10.1145/2629697

Siemens Digital Industries Software. 2026. Parasolid 3D Geometric Modeling. https://plm.sw.siemens.com/en-US/plm-components/parasolid/. Accessed: 2026-01-19.

Spatial Corp. 2026. CGM Modeler. https://www.spatial.com/solutions/3d-modeling/cgm-modeler. Accessed: 2026-01-19.

Yuqing Wang, Xiaohong Jia, Jieyin Yang, Bolun Wang, Pengbo Bo, and Yang Liu. 2025. Improving the Watertightness of Parametric Surface/Surface Intersection. *Computer Graphics Forum* (Dec. 2025). doi:10.1111/cgf.70298

Zheng Wei and Xiaodong Wei. 2024. Scalable Field-Aligned Reparameterization for Trimmed NURBS. arXiv:2410.14318 [cs.CG] https://arxiv.org/abs/2410.14318

Huibiao Wen, Guilong He, Rui Xu, Shuangmin Chen, Shiqing Xin, Zhenyu Shu, Taku Komura, Jieqing Feng, Wenping Wang, and Changhe Tu. 2025. Feature-Preserving Mesh Repair via Restricted Power Diagram. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers (SIGGRAPH Conference Papers '25)*. ACM, 1–11. doi:10.1145/3721238.3730671

Karl DD Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G Lambourne, Armando Solar-Lezama, and Wojciech Matusik. 2021. JoinABLe: Learning Bottom-up Assembly of Parametric CAD Joints. *arXiv preprint arXiv:2111.12772* (2021).

X. Xiao, P. Alliez, L. Busé, and L. Rineau. 2021. Delaunay Meshing and Repairing of NURBS Models. *Computer Graphics Forum* 40, 5 (Aug. 2021), 125–142. doi:10.1111/cgf.14362

Xin Yang, Fei Yu, Yan Zhou, Pengchao Zhou, G.Z. Zhao, and Z.Q. Guan. 2025a. A method of surface mesh generation for industrial CAD models by constructing conforming discrete representation. *Computer-Aided Design* 188 (Nov. 2025), 103914. doi:10.1016/j.cad.2025.103914

Xin Yang, Fei Yu, Yan Zhou, Pengchao Zhou, G.Z. Zhao, and Z.Q. Guan. 2025b. A method of surface mesh generation for industrial CAD models by constructing

conforming discrete representation. *Computer-Aided Design* 188 (Nov. 2025), 103914. doi:10.1016/j.cad.2025.103914

Fei Yu, Jie Cao, Julin Shan, S. H. Lo, and Zhenqun Guan. 2021. <scp>PASM</scp>: <scp>Parallel</scp> aligned surface meshing. *Internat. J. Numer. Methods Engrg.* 122, 15 (April 2021), 3705–3732. doi:10.1002/nme.6678

Y. Zheng, N.P. Weatherill, and O. Hassan. 2001. Topology Abstraction of Surface Models for Three-Dimensional Grid Generation. *Engineering with Computers* 17, 1 (May 2001), 28–38. doi:10.1007/s003660170021

Daniel Zint, Zhouyuan Chen, Yifei Zhu, Denis Zorin, Teseo Schneider, and Daniele Panozzo. 2025. Topological Offsets. arXiv:2407.07725 [cs.CG] https://arxiv.org/abs/2407.07725